```
# stap ip_input.stp
ip_input-> time=1320311892 funcion = ip_rcv
ip_input-> time=1320311892 funcion = ip_rcv_finish
ip_input-> time=1320311892 funcion = ip_local_deliver
ip_input-> time=1320311892 funcion = ip_local_deliver_finish
ip_input-> time=1320311892 funcion = ip_rcv
```

Probing function return value

To probe the return value of a function, *'$$return'* is used. The *return* value will be in a string. It can only be retrieved from a return event. The syntax is given below:

```
probe kernel.function("function-name").return {}
probe syscall.syscall-name {}
```

Test script:

```
# cat return.stp
probe syscall.mkdir.return
{
      printf ("mkdir() %s\n",$$return);
}
```

We have chosen the *'mkdir()'* system call (called whenever a directory is created). Execute the script, and create a directory (run *mkdir test_dir* in another terminal):

```
# stap return.stp
...
mkdir() return=0x0
```

The *mkdir* system call returns 0 on success, which has been printed by the script.

Conditional statements

In SystemTap scripts, like other programming languages, we can use *if* and *if-else* statements for branching on conditions. The syntax is simple:

```
if(condition)
      statement1
else
      statement2
```

Example:

```
# cat ifelse.stp
global counter
probe kernel.function("*@net/ipv4/ip_input.c")
{
      if (probefunc() == "ip_rcv")
      counter++;
}
probe timer.s(5)
{
      exit();
}
probe end
{
      printf ("ip_rcv() has been called %d times\n", counter);
}
```

In this script, we declared a global variable and incremented it every time *ip_rcv()* was called. After 5 seconds, the timer handler will be called and it will executive exit() function which in turn will exit the script. At the end, the *end* handler will print the counter:

```
# stap ifelse.stp
ip_rcv() has been called 12 times
```

This type of script can also be used by application developers who want to monitor kernel work when an application runs in user space.

Creating functions

Reusing source code by creating functions can also be done in SystemTap. Functions also make our scripts easy to control and read. The syntax to create a function is *function function_name(arguments) {statements}* and to use a function within a handler, it is simply *function_name(arguments)*:

```
# cat func.stp
function myfunc(){
      printf("my function\n");
}
probe begin
{
      myfunc();
      printf ("hello world\n");
}
```

Let us call our simple function *myfunc()* from an event handler. The output is as expected:

```
# stap func.stp
my function
hello world
```

There are several other options in SystemTap which you can explore. For more information, refer to *http://sourceware.org/systemtap/documentation.html*. END

By: Manoj Kumar

The author is a freelance developer and trainer. His areas of interest are the Linux kernel, Linux administration and Qt application development. You can reach him at *manoj@nxtsolutions.com* and visit his blog, *www.open4india.blogspot.com*, to share your Linux knowledge.